

DATA ARCHITECT ACADEMY

18 SEP 2026

TECHNICAL FOUNDATIONS - TF1

Understand architecture as a system of trade-offs driven by business constraints and long-term consequences.

About me

Iván Trebilcock

Senior Architect | Data & AI



- **What I do**

I work across data architecture, cloud platforms, data engineering, and AI – turning business problems into architectures that teams can actually build and operate.

- **What I've learned**

Architecture is rarely about finding the “perfect” technology. It's about understanding the problem, asking the right questions, making trade-offs, and communicating decisions.

- **Why I'm here**

I created this Academy to share the lessons that have helped me – and to learn together by applying them to realistic problems.

What This Course Is About

DA -Academy prepares data professionals to become confident, AI-Enabled architects and leaders

- This course is designed to close the foundational gaps many of us bring with us.
- We will start from the fundamentals so you can understand how to think about and design data engineering solutions.

Learning objectives

The module builds from understanding to practice, then toward judgment.

Understand

- Architecture is about decisions and trade-offs, not frameworks or diagrams
- Functional requirements vs. quality attributes
- Architecturally Significant Requirements (ASRs) and cost of change

Apply

- Identify business objectives and constraints for a scenario
- Write measurable quality attribute scenarios
- Build a Utility Tree; trace decisions to trade-offs

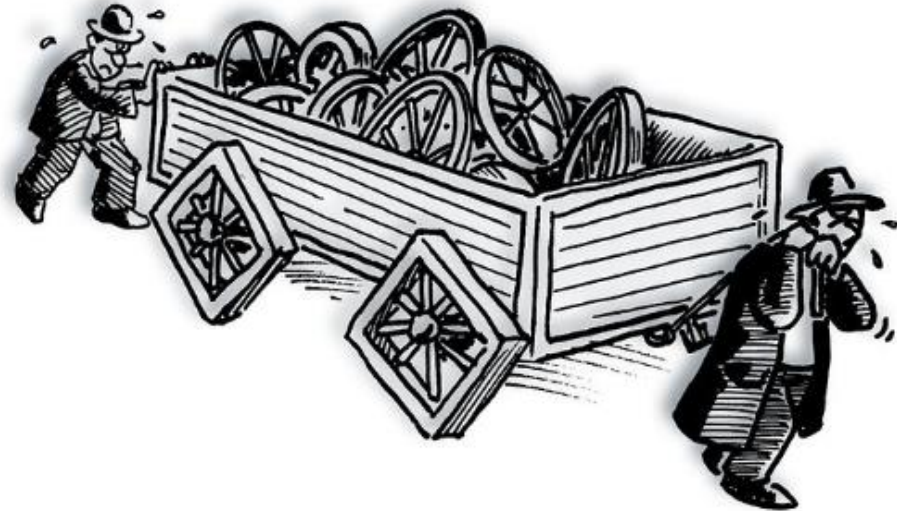
Evaluate

- Assess whether a decision is justified by objectives and constraints
- Identify hidden trade-offs in existing architectures
- Facilitate a team through the objectives → decisions chain

Tools Don't Replace Judgment

- **Know the tool**

Understand what it solves – and what it doesn't.

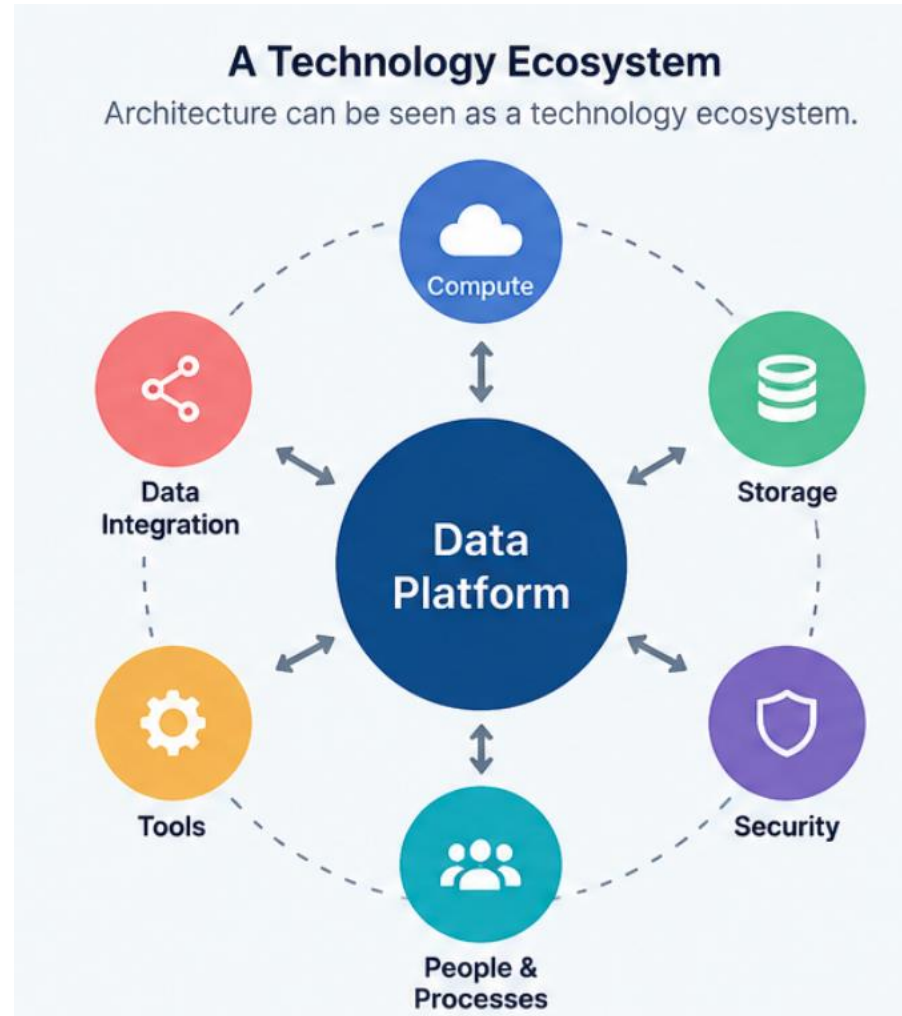


Agenda

- What is Architecture
- Architecture as Decision-Making
- Objectives, Constraints, and Requirements
- Quality Attributes as Decision Drivers
- From Objectives to Architectural Decisions
- ASRs to Decisions
- Scenario A: Retail IoT Monitoring Platform
- Utility Trees and ASR Prioritization
- Cost of Change

What is Architecture?

More than tools. It's a system of decisions that defines what your system can do



What is Architecture?

More than tools. It's a system of decisions that defines what your system can do

- **Defines the system's needs**

Architecture translates business objectives, constraints and requirements into a coherent system.

- **Architectural decisions are strategies**

They are long-term choices that shape the capabilities of the system, not just the implementation details.

- **It's about capabilities**

Architecture describes what the system can do, how well it can do it, and how it can evolve.

Choosing the right tools requires judgment

There are general-purpose tools and specialized tools. The key is to understand the problem you are solving and select the right tool for the job.

Level of granularity matters

Even decisions like "What is the right level of granularity of a table?" are architectural, because they impact performance, cost, scalability and future flexibility.

Architecture is decision-making

The job is not to pick fashionable tools. The job is to make justified trade-offs.

- Architecture is the set of important design decisions that shape a system.

Important decisions

Structure, performance, scalability, maintainability, security, and cost of change.

Trade-offs

Monolith vs. microservices, batch vs. streaming, SQL vs. NoSQL, sync vs. async.

Long-term evolution

Architecture determines how systems grow, adapt, and survive change.

Inputs to architectural design

Understand the problem before proposing the solution.

Business objectives

The value the system must deliver: revenue, CX, data volume, real-time analytics.

Constraints

Limits on the solution space: budget, timeline, regulation, infrastructure, expertise.

Functional requirements

What the system must do: create orders, store data, generate reports, process payments.

Quality attributes

How well the system performs: performance, scalability, availability, security, modifiability.

Functional vs. Quality requirements

Functional requirements say what happens. Quality attributes say how well it happens.

- Functional – what the system does
 - Create orders
 - Store data
 - Generate reports
 - Process payments
- Quality – how well it performs
 - Response < 1s
 - 99.9% availability
 - Scale 10x without re-architecture
 - PII policies enforced centrally

Quality attributes as architecture drivers

Quality attributes directly shape architectural choices.

Performance

Caching, async processing, load balancing

Scalability

Distributed architecture, event streaming

Availability

Redundancy, failover, replication

Security

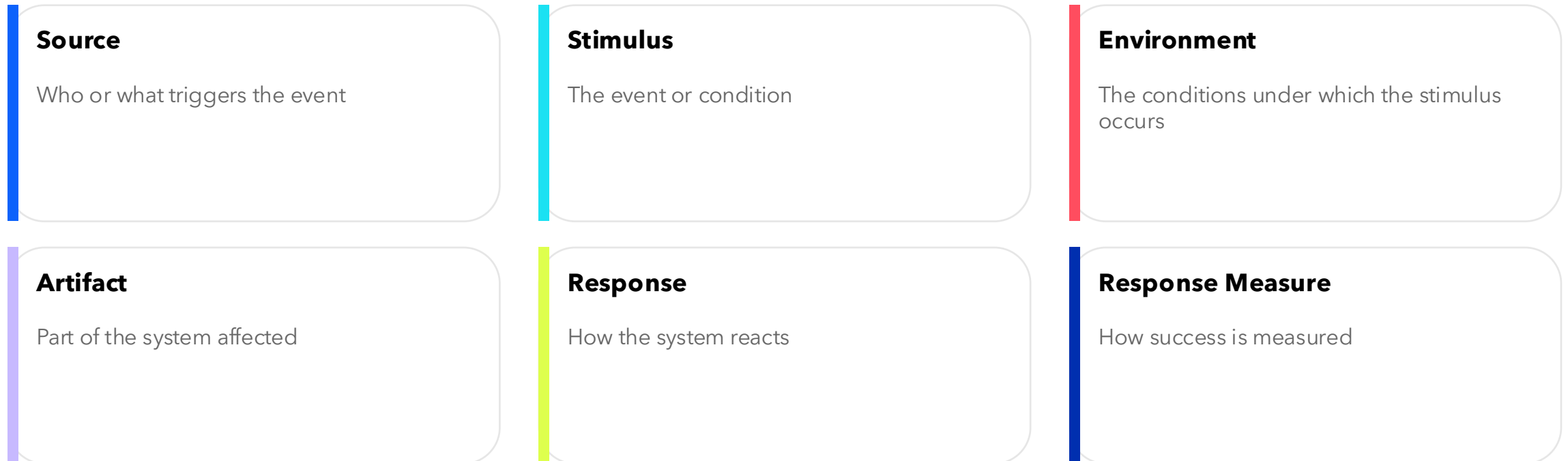
Boundary controls, policy enforcement, audit logs

Modifiability

Layering, decoupling, schema evolution

Make quality attributes measurable

Use the scenario structure before choosing architecture.



- Example measure: p95 ingestion latency < 200ms with zero event loss.

Architecturally Significant Requirements

Not every requirement has the same impact on architecture.

An **Architecturally Significant Requirement (ASR)** is a requirement that materially influences the structure, behavior, or major design decisions of the system.

Scale pressure

Support 10M concurrent users or 50K events per second.

Recovery pressure

Recover in less than 30 seconds or meet RTO/RPO commitments.

Compliance pressure

Strict data security, access control, masking, and auditability.

Scenario A: Retail IoT Monitoring Platform

Business Context

A retail company operates **500 stores** equipped with IoT devices that continuously generate telemetry events.

Operations teams want to use this data to **monitor store and device conditions, detect operational issues, and support near-real-time analytics.**

The company expects the number of connected devices to grow as the platform expands to new regions.

What the system must do

The platform must:

- Receive telemetry events from devices across all stores
- Validate incoming events
- Persist accepted events for downstream processing
- Make the data available to analytics and operational applications
- Support new versions of device event schemas over time

Scenario A: Retail IoT Monitoring Platform

Operating Conditions

50K events/sec

Peak ingestion bursts during business hours

<200ms

p95 ingestion latency during peak load

<30 sec

Recovery from a single availability-zone failure

<1 day

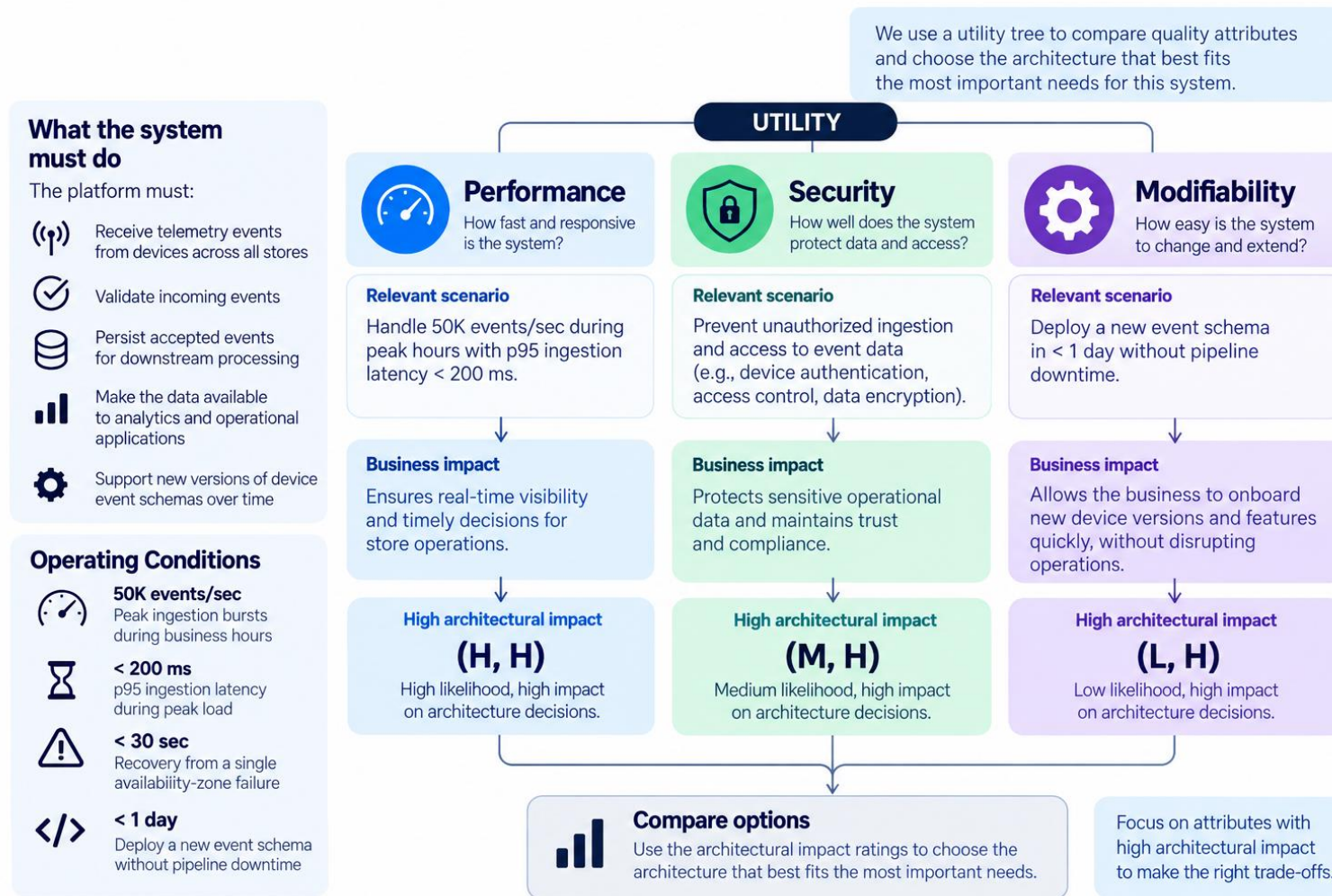
Deploy a new event schema without pipeline downtime

Additional Context

- Events may contain **device, store, customer, or loyalty identifiers** that require controlled access.
- Device firmware evolves independently and may introduce **new event schema versions**.
- The delivery team has **strong batch-processing experience but limited streaming experience**.

Utility tree: what cannot be wrong?

Scenario A's



A Utility Tree does not tell you what to build. It tells you what you cannot afford to get wrong.

The cost of change

Reversible

Table naming, dashboard layout, report format.

Partially reversible

Orchestration tool or cloud region.

Near-irreversible

Storage architecture, schema design, coupling between layers.

The Cost of Change Curve

The later a problem is discovered, the more expensive it is to fix.



The harder a decision is to reverse, the more intentional we should be about making

Architecture Must Evolve



Business needs, constraints, and our understanding of the problem evolve over time. Architecture should support that evolution – but important system capabilities rarely emerge by accident.

Design Upfront for Critical Capabilities

Identify the capabilities the system must enable, especially those that would be expensive to introduce later.

Examples: scalability, security boundaries, recoverability, interoperability, data isolation.

Allow the Design to Evolve

Not every implementation detail needs to be decided upfront.

As we learn, components, patterns, technologies, and implementation choices can evolve.

Architecture Must Evolve



Preserve Optionality

When uncertainty is high, avoid unnecessary irreversible decisions.
Create enough architectural structure today to keep tomorrow's important options open.

Continuously Reassess

As business objectives, constraints, and quality attributes change, revisit the architecture and its assumptions.

Trace the reasoning chain

Every significant decision should be traceable back to the business objective.



- Skip a step and the decision loses its justification.
- Quality attributes become useful when they are specific and measurable.
- ASRs narrow the design space and expose the trade-offs worth discussing.

Capstone scenario: Customer 360

Your challenge

A retail company wants to create a **Customer 360 data capability** that combines customer information and interactions across multiple systems.

The goal is to provide business and data teams with a **trusted view of the customer** while supporting analytics, personalization, customer service, and future Data Science use cases.

Business Context

- Customer information is fragmented across:
- E-commerce orders
- Loyalty program activity
- Customer support tickets
- Marketing interactions

Business teams currently spend significant time reconciling inconsistent customer information across systems.

Business needs

- **Marketing:** better segmentation and personalization
- **Customer Service:** visibility into recent customer interactions
- **Leadership:** trusted daily customer, engagement, and sales metrics
- **Data Science:** reliable historical data for churn and propensity models

Current Scale

- 20M customers
- 5M interactions/day
- 4 initial source systems
- 3 years of historical data

Capstone scenario: Customer 360

Operating conditions

- Daily dashboards must be ready by **7:00 AM**
- Marketing data can tolerate **several hours of latency**
- Customer Service interactions available within **15 minutes**
- Customer data contains **PII and sensitive information**
- Customer interactions expected to grow **3x within 2 years**
- **2-4 new sources/year** expected; a standard source should be onboarded within **2 weeks**
- Customer data contains **PII and sensitive information** requiring controlled and auditable access
- First production release required within **4 months**
- Infrastructure and processing **costs must remain observable as usage grows**

What the platform must do

- Ingest customer data from multiple source systems
- Validate and standardize incoming data
- Resolve customer identities across systems
- Preserve historical customer information
- Make trusted customer data available to analytics
- Support Data Science workloads
- Protect sensitive customer information
- Allow new sources and attributes to be added over time

Capstone scenario: Customer 360

Your task

Produce:

2-3 objectives → key constraints → functional and quality requirements → 2 quality-attribute scenarios → 3 ASRs → 3 architecture decisions → trade-offs

Not everything is defined. Do not invent requirements. Identify important gaps and state what you would ask the stakeholder.

Practice the full reasoning chain

Start with the problem. End with architecture. Be able to explain every decision in between.

Identify architectural inputs

1

Write quality attribute scenarios

2

Build a Utility Tree

3

Trace ASRs to decisions

4

Reflect on cost of change

5

Capstone: Customer 360 platform

6

Key takeaways

Great architects do not start with solutions. They start by understanding the problem space.



- Understanding business objectives



- Respecting constraints



- Designing for quality attributes



- Making deliberate trade-offs



- Controlling long-term cost of change

Thank you!

Architecture is justified decision-making under constraint.